

Stemmestyring for sanntidsinteraksjon: En dyp læringsbasert tilnærming

[GitHub](#) | [Applikasjon](#) | [YouTube Demo](#)

Kandidatnummer:

423 - 437 - 366 - 361

Faglærer: Steffen Mæland

Innleveringsdato: 30. april 2026

Innholdsfortegnelse

1 Innledning	1
1.1 Motivasjon og problemstilling	1
1.2 Begrunnelse for bruk av dyp læring	1
1.3 Prosjektets bidrag og avgrensning	2
2 Data	4
2.1 Datasettet	4
2.2 Målklasser og label-mapping	5
2.3 Klassefordeling og håndtering av ubalanse	5
2.4 Preprosessering og input-representasjon	6
2.5 Dataaugmentering	9
3 Metode og modellarkitektur	11
3.1 Modellarkitekturer	11
3.2 Treningsoppsett	12
3.3 Eksperimentoversikt	13
3.4 Implementasjonsmiljø og reproduserbarhet	13
4 Resultater og evaluering	14
4.1 Evalueringsmetodikk	14
4.2 Kvantitative resultater	14
4.3 Eksperimentsammenligning og diskusjon	15
4.4 Feilanalyse	16
4.5 Latency, modellstørrelse og sanntidskrav	16
5 Deployment	17
5.1 Systemarkitektur og applikasjonsdesign	17
5.2 Modellvalg og konfidensterskel	18
5.3 Begrensninger og videre arbeid	18
6 Konklusjon	19
7 Kilder	20
8 Vedlegg	21
8.1 Erklæring om bruk av kunstig intelligens (KI)	21

1 Innledning

1.1 Motivasjon og problemstilling

Interaksjon med dataspill og interaktive systemer foregår tradisjonelt gjennom tastatur, mus eller dedikerte kontrollere. Disse inndatametodene er veletablerte, men forutsetter at brukeren har frie hender og full motorisk kontroll. Talebasert styring representerer et alternativ som kan gjøre interaksjon mer tilgjengelig for brukere med nedsatt motorikk, og åpner for naturlige grensesnitt i situasjoner der manuell betjening er upraktisk.

Problemet hører hjemme i feltet *limited vocabulary speech recognition*, også omtalt som *keyword spotting*. I motsetning til fullskala talegjenkjenning, der systemet må håndtere fritt naturlig språk, begrenses oppgaven her til et lite, lukket vokabular. Denne avgrensningen er ikke bare en forenkling, men en forutsetning for at modellen kan kjøres lokalt med lav forsinkelse på ressursbegrenset maskinvare (Rybakov et al., 2020).

Prosjektets overordnede problemformulering er:

«Kan et kompakt, lokalt CNN-basert keyword spotting-system, trent fra bunnen av på mel-spektrogrammer fra Google Speech Commands-datasettet, oppnå klassifikasjonsnøyaktighet og inferenstid som er gode nok til å fungere som en praktisk og responsiv styringsenhet for et sanntids Snake-spill?»

Snake spillet fungerer her som en kontrollert testapplikasjon: spillet krever et minimum av fire tydelige direksjonelle kommandoer (*up, down, left, right*), feilklassifisering har umiddelbart synlige konsekvenser, og kravene til responstid er strenge nok til at både modell- og systemlatency må vurderes. Dette gjør det velegnet til å teste om en deep learning pipeline holder i praktisk bruk, ikke bare på et benchmark.

1.2 Begrunnelse for bruk av dyp læring

Lydsignaler er høydimensjonale og tidsvarierende: ett sekunds opptak ved 16 kHz samplingrate gir 16 000 datapunkter. Tradisjonelle metoder som Hidden Markov-modeller kombinert med håndlagde features (typisk MFCC) krever betydelig domenekunnskap og gir begrenset generaliseringsevne på tvers av ulike talere, uttaler og støynivåer. Dyp læring, og

særlig konvolusjonelle nevrale nettverk (CNN) har vist seg å overkomme disse begrensningene ved å lære relevante representasjoner direkte fra data (Goodfellow et al., 2016).

Ved å representere lydsignalet som et mel spektrogram, et todimensjonalt bilde av frekvens representasjon der frekvensen er skalert etter den menneskelige hørselens ikke-lineære oppløsning (Stevens et al., 1937), kan en CNN utnytte lokale mønstre i tid og frekvens på samme måte som bildeklassifisering (Chollet & Watson, 2026, kap. 8). Modellen lærer selv å ekstrahere relevante features uten manuell feature engineering, noe som gir bedre robusthet mot variasjon i stemme, volum og bakgrunnsstøy.

Andre arkitekturklasser ble vurdert og forkastet av eksplisitte grunner. Rekurrente nettverk (LSTM, GRU) gir høyere inferenstid og er vanskeligere å trene stabilt på begrensede datasett. Selvsupervised forhåndstreinte modeller som Wav2Vec 2.0 oppnår høyere ytelse på generelle taleoppgaver, men forutsetter store mengder umerkede data og betydelig regnekraft, og bryter med kursets krav om at minst en modell skal trenes fra bunnen av. En kompakt CNN på mel spektrogram representerer derfor det beste kompromisset mellom ytelse, trenbarhet og inferenshastighet for et lite, lukket vokabular (Rybakov et al., 2020).

1.3 Prosjektets bidrag og avgrensning

Prosjektets bidrag er å demonstrere en komplett ende-til-ende deep learning-pipeline, fra datainnhenting og preprosessering, via systematisk eksperimentering med arkitektur, ubalans håndtering, til en deployert interaktiv applikasjon som kjøres lokalt uten skytjenester. Alle modeller er trent fra bunnen av med tilfeldig vektinitialisering, i tråd med kursets krav.

Suksess vurderes langs tre dimensjoner:

(i) **klassifikasjonsytelse** måles på det offisielle testsettet med macro F1-score som primærmetrikk og accuracy som sekundærmetrikk. Macro F1 er valgt fordi 5-klasse oppsettet er sterkt ubalansert (se kap. 2 *Data*), og en metrikk som vekter klassene likt gir et mer ærlig bilde av modellens evne til å skille kommandoer fra irrelevant tale.

(ii) **Inferenstid** skal være godt under 100 ms per klipp, som er den øvre grensen for det brukere oppfatter som umiddelbar respons i interaktive systemer.

(iii) **Funksjonell spillbarhet** vurderes kvalitativt gjennom en deployert Snake demo der modellen styrer spillet i sanntid via mikrofonen.

Prosjektet avgrenses til engelske kommandoer fra Google Speech Commands-datasettet og inkluderer ikke en egen silence-klasse; konsekvensene av begge avgrensningene diskuteres i kapittel 5 *Deployment* og 6 *Konklusjon*.

2 Data

2.1 Datasettet

Prosjektet benytter Google Speech Commands v0.02 (Warden, 2018), tilgjengelig via HuggingFace Datasets. Tabell 1 gir en oversikt over de viktigste egenskapene ved datasettet.

Egenskap	Verdi
Totalt antall lydklipp	105 000
Antall originale kommandoklasser	36
Samplingrate	16 000 Hz (16 kHz)
Klippelengde	1 sekund
Unike talere (treningssett)	> 2 000
Lyd-format	Mono WAV
Kilde	HuggingFace Datasets (google/speech commands v0.02)

Tabell 1: Oversikt over Google Speech Commands v0.02.

Datasettet er et etablert benchmark for *limited vocabulary speech recognition*. Treningssettet inkluderer over 2 000 unike talere. Dette gir variasjon i stemme og dialekt og reduserer overfitting til enkelttalere.

Datasettet har forhåndsdefinerte train/validation/test-splitter med talerne separert mellom splittene; en verifisering avdekket under 0,05 % overlapp. Dette sikrer evaluering på ukjente stemmer og direkte sammenlignbarhet med litteratur som bruker samme splitter. En sentral styrke ved datasettet er de forhåndsdefinerte train/validation/test-splittene, der talerne er eksplisitt separert mellom splittene. En verifisering avdekket én overlappende taler-ID per splittpar, tilsvarende under 0,05 % av alle talere, som regnes som neglisjerbart. Dette sikrer at modellen evalueres på helt ukjente stemmer og at resultatene er direkte sammenlignbare med eksisterende litteratur som benytter de samme splittene.

2.2 Målklasser og label-mapping

Snake-spillet krever kun fire styringskommandoer. Originalsettet med 36 klasser ble remappet til en forenklet målstruktur: de fire kommandoene (up, down, left, right) beholdes som egne klasser, mens samtlige øvrige kommandoer samles i en felles klasse kalt «unknown».

Mappingen er implementert eksplisitt i kode og lagret som egne target-felt i en reproduserbar index.csv-fil.

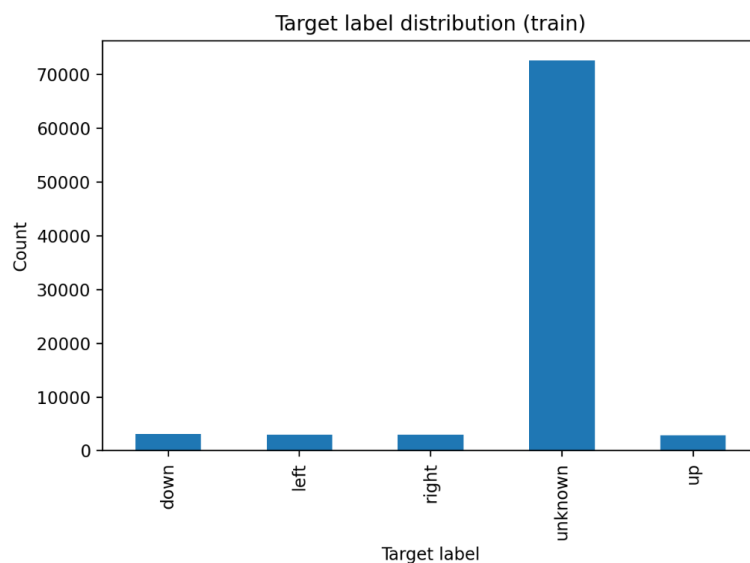
Remappingen gir to eksperimentoppsett: et balansert 4-klasse-oppsett (kun de fire kommandoene) som fungerer som referansebaseline, og et realistisk 5 klasse oppsett (inkludert unknown) som simulerer faktisk brukssituasjon der systemet også må håndtere uønsket tale og bakgrunnsstøy.

2.3 Klassefordeling og håndtering av ubalanse

Tabell 2 viser antall eksempler per kommandoklasse i hvert datasett split. Etter remapping oppstår en markant klasseubalanse: de fire kommandoklassene har om lag 3 000 treningseksempler hver, mens «unknown» dominerer med om lag 72 000 eksempler, tilsvarende ca. 86 % av treningsdataene, som illustrert i Figur 1.

Split	Down	Left	Right	Up	Unknown
Train	3 000	3 000	3 000	3 000	72 000
Validation	400	400	400	400	9 600
Test	400	400	400	400	9 600

Tabell 2: Antall eksempler per kommandoklasse i hvert datasett-split (tilnærmede verdier).



Figur 1: Klassefordeling i treningssettet etter remapping fra 36 til 5 klasser. «Unknown»-klassen dominerer med 86 % av dataene og må håndteres eksplisitt under trening.

Denne skjevheten gjør accuracy til en utilstrekkelig metrikk, ettersom en modell som alltid predikerer «unknown» ville oppnå høy accuracy uten å gjenkjenne noen kommando. For å motvirke ubalansen kombineres to tiltak: class-weighted CrossEntropyLoss i grunnleggende 5 klasse-trening, der vektene er inverselt proporsjonale med klassefrekvens, og eksplisitt nedprøving (downsampling) av «unknown» til maksimalt 12 000 treningseksempler i eksperiment E1 og E2. Validerings- og testsett holdes alltid uendret for å sikre evaluering på reell fordeling.

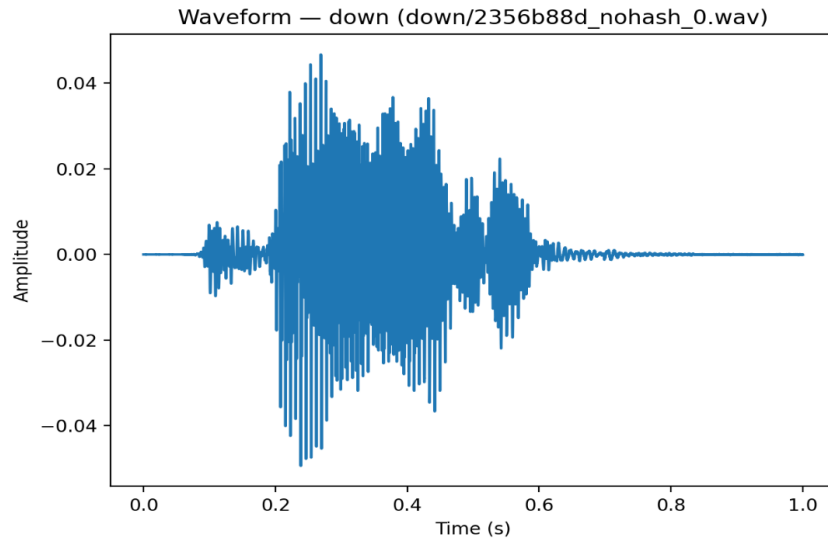
2.4 Preprosessering og input-representasjon

All audio normaliseres til eksakt ett sekund, tilsvarende 16 000 samples: klipp kortere enn dette zero paddes i halen, mens lengre klipp trunkeres. Tabell 3 gir en fullstendig oversikt over preprosesseringsparameterne.

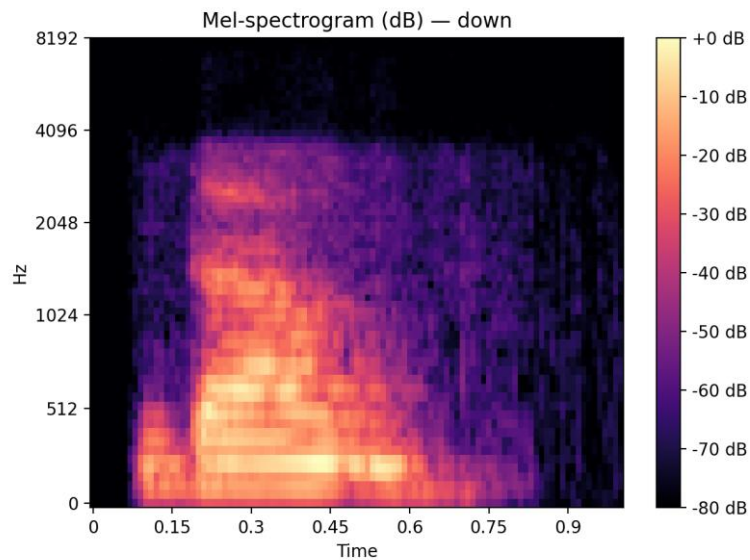
Parameter	Verdi
Samplingrate	16 000 Hz
Klipp lengde (normalisert)	1 sekund (16 000 samples)
Håndtering av for korte klipp	Zero-padding (hale)
Håndtering av for lange klipp	Trunkering
Antall mel-bins	64
FFT-vindustørrelse (n_fft)	400 samples (25 ms)
Hop-lengde	160 samples (10 ms)
Output tensor-form	(1, 64, 101) per sample
Bibliotek	torchaudio (MelSpectrogram, AmplitudeToDB)

Tabell 3: Parametere for preprosessering og mel-spektrogram-ekstraksjon.

Mel skalaen er valgt fremfor lineær frekvensrepresentasjon fordi den etterligner det menneskelige hørselssystemets ikke lineære frekvensoppløsning og er dermed godt tilpasset talesignaler (Chollet & Watson, 2026, kap.8). Figur 2 og Figur 3 viser henholdsvis waveform og mel-spektrogram for et representativt eksempel av kommandoen «down». Mel-spektrogrammet viser tydelig at den akustiske energien er konsentrert i lave frekvenser og at talesignalet opptar et tydelig avgrenset tidsvindu. Dette mønsteret er typisk for vokalkommandoer og egner seg godt som input til en CNN.



Figur 2: Waveform for kommandoen «down». Talesignalet er avgrenset i tid og omgitt av stillhet som følge av zero-padding-politikken.



Figur 3: Mel-spektrogram (dB) for kommandoen «down», med form (1, 64, 101). Energikonsentrasjonen i lave frekvenser og det tydelige temporale mønsteret er karakteristisk for vokalkommandoer.

For den alternative Raw Waveform-arkitekturen brukes den normaliserte waveformen direkte med form (1, 16 000), uten spektrogramtransformasjon. Dette muliggjør en direkte sammenligning av de to input-representasjonene under ellers identiske betingelser.

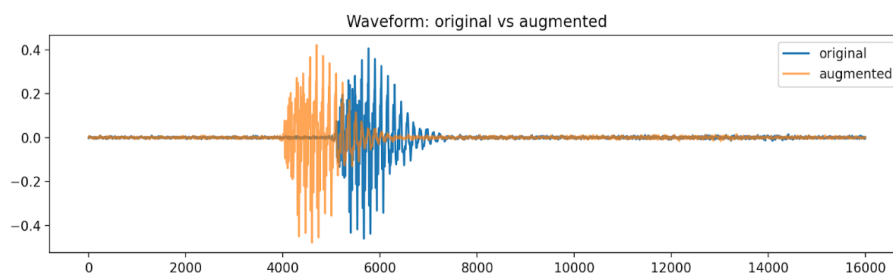
2.5 Dataaugmentering

I eksperiment E2 suppleres treningsdataene med *on the fly* augmentering for å øke modellens robusthet mot variasjoner som oppstår ved live mikrofon input. Tre teknikker benyttes, som vist i Tabell 4. All augmentering gjøres kun på treningsdataene; validerings- og testdata forblir alltid uendret.

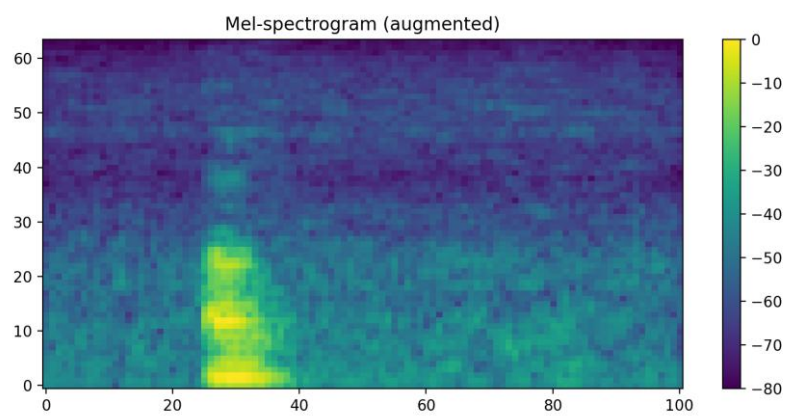
Teknikk	Beskrivelse	Formål
Gain justering	Tilfeldig skalering av amplitude	Simulerer variasjon i mikrofonvolum
Tidsforskyvning	Syklisk forskyvning av signalet i tid	Simulerer variasjon i taletiming
Additiv gaussisk støy	Tilfeldig støy adderes til signalet	Simulerer bakgrunnsstøy i omgivelsene

Tabell 4: Oversikt over augmenteringsteknikker brukt i eksperiment E2.

Figur 4 viser effekten av augmentering på waveform nivå, og Figur 5 viser tilhørende mel spektrogram etter augmentering. Det augmenterte spektrogrammet er klart mer støy-fylt, men bevarer de sentrale akustiske strukturene fra originalsignalet. Augmenteringen er implementert direkte i DataLoader pipelinen uten mellomlagring, og med faste tilfeldighetsfrø for reproduserbarhet.



Figur 4: Original waveform (blå) sammenliknet med augmentert versjon (oransje). Tidsforskyvning og gain-endring er tydelig synlig.



Figur 5: Mel-spektrogram etter augmentering. De sentrale frekvensstrukturene bevares, men støynivået er merkbart høyere.

3 Metode og modellarkitektur

3.1 Modellarkitekturer

To ulike arkitekturer er implementert og sammenlignet for å vurdere effekten av valg av input representasjon på ytelse og modellstørrelse. Begge er trent fra bunnen av med tilfeldig vektinitialisering. Tabell 5 oppsummerer de viktigste arkitekturegenskapene.

Komponent	SmallSpectrogramCNN	RawWaveformConv1D
Input	Mel-spektrogram ($1 \times 64 \times 101$)	Rå lydsignal ($1 \times 16\,000$)
Kjernlag	Conv2D $\times 3$, MaxPool, BatchNorm	Conv1D $\times 3$, GlobalAvgPool, BatchNorm
Aktivering	ReLU	ReLU
Regularisering	Dropout, BatchNorm	Dropout, BatchNorm
Klassifikator	Linear (64 til N klasser)	Linear (64 til N klasser)
Antall parametere	~23 780	~9 621
Checkpoint-størrelse	< 1 MB	< 1 MB

Tabell 5: Sammenligning av de to modellarkitekturene.

SmallSpectrogramCNN er prosjektets primærarkitektur og behandler mel spektrogrammet som et 2D bilde. Nettverket består av tre konvolusjonslag (Conv2D) med påfølgende batchnormalisering, ReLU-aktivering og MaxPooling, etterfulgt av et fullt tilkoblet klassifikasjonshode med dropout. Tre lag er valgt i tråd med etablert praksis for kompakte keyword-spotting-modeller (Zhang & Zhang, 2020; Rybakov et al., 2020), og gir et reseptivt felt som dekker hele mel-spektrogrammet samtidig som parametertallet holdes under 25 000 og inferenstiden under sanntidskravet. En systematisk hyperparameter-optimalisering av

nettverksdybde lå utenfor prosjektets rammer. Batchnormalisering stabiliserer treningsprosessen og reduserer sensitiviteten overfor læringsraten (Goodfellow et al., 2016). Arkitekturen er holdt kompakt med om lag 23 780 parametere for å holde inferenstiden lav og muliggjøre lokal kjøring uten GPU. SmallSpectrogramCNN er valgt som prosjektets deployerte modell i Snake-demoen, se kapittel 5.

RawWaveformConv1D er en alternativ arkitektur der nettverket mottar det rå lydsignalet i tidsdomenet direkte, uten forhåndsberegnet spektrogram. Temporale mønstre ekstraheres gjennom tre Conv1D lag etterfulgt av global gjennomsnittspooling, som reduserer sekvensen til en fast vektorrepresentasjon uavhengig av inndatalengde. Modellen er enda mer kompakt med om lag 9 621 parametere, men mangler den eksplisitte tids frekvens representasjonen som mel spektrogrammet tilbyr. Sammenligningen er direkte ettersom de to arkitekturene evalueres under identiske eksperimentbetingelser.

3.2 Treningsoppsett

Begge arkitekturer trenes med Adam optimizer (Kingma og Ba, 2015) og CrossEntropyLoss. Adam er valgt fordi det kombinerer adaptiv læringsrate per parameter med momentumakkumulering, noe som gir rask og stabil konvergens uten behov for manuell justering av læringsraten over tid. I 5-klasse-variantene benyttes class-weighted CrossEntropyLoss, der vektene er inverselt proporsjonale med klassefrekvens i treningssettet, for å kompensere for dominansen til «unknown»-klassen.

Læringsrate, batch størrelse og antall epoker holdes konsistente på tvers av sammenlignbare eksperimenter, slik at effekten av individuelle endringer kan isoleres. Konkret brukes læringsrate $1 \cdot 10^{-3}$ og ti epoker i alle eksperimenter, mens batch-størrelsen er tilpasset hvert oppsett: 16 for 4-klasse-baseline, 32 for E1 og E2 (downsampling og augmentering) og 64 for 5-klasse-weighted og Raw Waveform. Mindre batcher gir mer stokastisk variasjon og er valgt der augmentering og ubalansebehandling krever fleksibilitet, mens større batcher gir raskere konvergens i enklere oppsett. Det beste checkpointet lagres basert på validation accuracy, og all videre evaluering benytter denne modellen. For 5-klasse-eksperimentene ville validation macro F1 vært en mer robust seleksjonsmetrikk gitt klasseubalansen, men i praksis sammenfaller de beste epokene for begge metrikker, og effekten på sluttytelsen vurderes som liten. Tap og accuracy per epoke logges til TensorBoard for visuell sporing og reproducerbar eksperimentlogging.

3.3 Eksperimentoversikt

Fem eksperimenter er gjennomført i en systematisk rekkefølge for å isolere effekten av hvert enkelt designvalg. Tabell 6 gir en kompakt oversikt.

Eksperiment	Klasser	Input	Ubalansehåndtering	Augm.
Baseline 4-klasse	4	Mel-spektrogram	Ingen (balansert)	Nei
Baseline 5-klasse	5	Mel-spektrogram	Class-weighted loss	Nei
E1: Downsampling	5	Mel-spektrogram	Downsampling unknown	Nei
E2: Downsampling og aug.	5	Mel-spektrogram	Downsampling unknown	Ja
Raw Waveform Conv1D	5	Rå waveform	Downsampling unknown	Nei

Tabell 6: Oversikt over gjennomførte eksperimenter.

3.4 Implementasjonsmiljø og reproduserbarhet

All kode er skrevet i Python med PyTorch og torchaudio. Trening ble gjennomført i Google Colab med GPU; demoen kjøres lokalt. Datasettet er indeksert i en index.csv med target-mapping for begge klasseoppsett. Tilfeldighetsfrø er satt for PyTorch og NumPy. Kode, checkpoints og figurer er tilgjengelig i et offentlig GitHub-repositorium.

4 Resultater og evaluering

4.1 Evalueringsmetodikk

Alle modeller ble evaluert på det offisielle testsettet fra Google Speech Commands, mens valideringssettet kun ble brukt til modellseleksjon. Dette reduserer risikoen for optimistiske ytelsesestimater og gjør sammenligningen mellom eksperimentene mer pålitelig.

To metrikker ble brukt: accuracy og macro F1-score. I 5-klasse-oppsettet er accuracy lite informativ alene fordi unknown dominerer; macro F1 vekter klassene likt og er derfor hovedmetrikk.

I tillegg ble modellene vurdert med tanke på praktisk bruk i sanntid. Her var særlig inferenstid, modellstørrelse og responsivitet i Snake-demoen viktige, siden en brukbar stemmestyringsløsning må være både nøyaktig og rask nok til å oppleves som naturlig.

4.2 Kvantitative resultater

Tabell 7 oppsummerer resultatene for alle eksperimenter. Den balanserte 4-klasse-CNN-modellen oppnådde 83,9 % test accuracy og 83,5 % test macro F1, noe som viser jevn ytelse på tvers av de fire kommandoene. Dette gjør den godt egnet som referansemødel og som grunnlag for deployment.

I 5-klasse-oppsettet blir oppgaven tydelig vanskeligere. Baseline-modellen med class-weighted loss oppnådde høy valideringsaccuracy, men langt lavere macro F1 på testsettet. Dette viser at accuracy alene gir et for positivt bilde når unknown er dominerende.

E1, der unknown ble nedprøvd i treningsdataene, ga lavere test accuracy enn baseline 5-klasse, men en klart bedre balanse mellom accuracy og macro F1. Dette tyder på at modellen i større grad lærte å skille de fire kommandoene fra unknown i stedet for å lene seg mot majoritetsklassen.

Den beste 5-klasse-modellen var E2, som kombinerte nedprøving og dataaugmentering. Denne oppnådde 78,4 % test accuracy og 70,7 % test macro F1, og ga dermed den sterkeste samlede ytelsen i det mest realistiske oppsettet.

Den alternative Raw Waveform Conv1D-modellen hadde lavest parameterantall, men presterte tydelig svakere enn spektrogrammodellen på macro F1. Resultatene tyder derfor på

at mel-spektrogram gir en mer hensiktsmessig representasjon enn rå waveform i dette prosjektet.

Modell	Param.	Val Acc	Val F1	Test Acc	Test F1	CPU (ms)
Baseline 4-klasse (CNN)	23 780	84,8 %	84,4 %	83,9 %	83,5 %	—
Baseline 5-klasse (vektet)	23 845	89,1 %	67,4 %	74,9 %	63,7 %	—
E1: Downsampling	23 780	75,4 %	58,7 %	69,1 %	68,6 %	—
E2: Downsampling og aug.	23 780	85,4 %	65,6 %	78,4 %	70,7 %	—
Raw Waveform Conv1D	9 621	87,6 %	49,7 %	70,9 %	45,2 %	0,84

Tabell 7: Fullstendig resultatoversikt for alle eksperimenter. Macro F1 er primærmetrikk. Grønn rad markerer de to beste modellene per oppsett.

4.3 Eksperimentsammenligning og diskusjon

Sammenlikningen mellom 4- og 5-klasse-oppsettene illustrerer hvor mye oppgaven endrer seg når modellen også må håndtere irrelevante ord. I det balanserte 4-klasse-oppsettet gir accuracy og macro F1 tilnærmet samme bilde, mens avviket mellom dem i 5-klasse-oppsettet avslører at modellen lener seg på unknown som «sikker» prediksjon.

At E1 forbedrer balansen og E2 ytterligere hever macro F1, tyder på at de to tiltakene virker på forskjellige problemer — nedprøving reduserer classeskjevheten i treningssignalet, mens augmentering utvider variasjonen modellen møter. Arkitektursammenlikningen peker i samme retning: mel-spektrogrammets eksplisitte tid-frekvens-struktur gir en induktiv fordel som Raw Waveform Conv1D måtte lure seg til fra rå tidssignal, noe modellen ikke klarte innenfor ti epoker og under 10 000 parametere.

4.4 Feilanalyse

Konfusjonsmatrisene viser to hovedtyper feil. I 4-klasse-oppsettet forveksler modellen akustisk like kommandoer, særlig right som klassifiseres som left. I 5-klasse-oppsettet er feilmønsteret asymmetrisk: down og right har lav recall (henholdsvis 38 % og 67 %) fordi mange ekte eksempler havner i unknown, mens up og left har høy recall (over 90 %) men lavere precision fordi unknown-eksempler lekker inn i disse klassene.

I Snake er en ignorert kommando mindre alvorlig enn en feil retning: det første oppleves som forsinkelse, det andre kan koste liv. Derfor bruker Gradio-appen en konfidensterskel på 0,55, som forkaster usikre prediksjoner på bekostning av noen tapte kommandoer.

4.5 Latency, modellstørrelse og sanntidskrav

Begge arkitekturer er kompakte med under 24 000 parametere og checkpoint under 1 MB, noe som muliggjør lokal kjøring uten kraftig maskinvare. Latency er målt eksplisitt for Raw Waveform Conv1D til 0,84 ms per inferens på CPU (batch=1, etter warmup).

SmallSpectrogramCNN har sammenlignbar parameterstørrelse og forventes å ligge i samme størrelsesorden, noe demoens responsivitet bekrefter. Rybakov et al. (2020) rapporterer tilsvarende latency for kompakte keyword-spotting-modeller, hvilket støtter prosjektets designvalg.

Total systemlatency er likevel høyere enn modellinferensen alene: lydopptak, preprosessering, grensesnitt og spilloppdatering bidrar alle. Demoen viser at totalresponsen forblir godt innenfor det interaktive kravet på 100 ms.

5 Deployment

5.1 Systemarkitektur og applikasjonsdesign

Den deployerte applikasjonen er implementert som en Gradio-nettapplikasjon med fire faner: en stemmeklassifiserer for isolert prediksjon med latency-breakdown, et manuelt Snake spill med knappkontroll, en stemmestyrte Snake-fane (ta-opp-og-utfør), og en live Snake-fane med kontinuerlig mikrofon-streaming. Applikasjonen kjøres lokalt og bruker E2-checkpointet (5-klasse, downsampling + augmentering).

I den stemmestyrte Snake-fanen følger pipelinen en fast sekvens for hver kommando:

- Lydklipp fanges via mikrofon eller filopplasting i Gradio-grensesnittet.
- Klippet normaliseres til 1 sekund og transformeres til mel-spektrogram ($1 \times 64 \times 101$) med samme preprosessering som under trening.
- Spektrogrammet sendes gjennom SmallSpectrogramCNN, som returnerer en sannsynlighetsfordeling over de fem klassene.
- Prediksjonen aksepteres kun dersom den predikerte klassen er en kommando og konfidensscoren overstiger 0,55; ellers ignoreres input.
- Snake-spillet tilstand oppdateres med den aksepterte kommandoen og ny ramme rendres med bildet.

Live fanen utvider denne pipelinen med tre tilleggskomponenter for å håndtere reell mikrofonbruk: en streaming buffer som akkumulerer 1 sekund med lyd kontinuerlig, en RMS basert noise gate (energi terskel 0,03) som forkaster opptak uten signifikant signal før de når modellen, og en autonom spillsløyfe som flytter slangen fremover hvert 0,5 sekund mens stemmen brukes til å snu venstre eller høyre. Snake spillet er implementert som en selvstendig klasse (snake_game.py) for å skille spillogikk fra applikasjonslogikk, og kommandoene tolkes som relative svinger som bufres til neste spilloppdatering.

5.2 Modellvalg og konfidensterskel

Den deployerte modellen er E2 (5-klasse), ikke 4-klasse-CNN-baseline som hadde høyere absolutt macro F1 på sine respektive klasser. Valget er bevisst: i reell bruk vil mikrofonen fange opp bakgrunnstale, pust og uvedkommende lyder. En modell uten *unknown* klasse ville da være tvunget til å klassifisere alt som en av de fire retningene. E2 håndterer dette eksplisitt ved at irrelevante signaler kan klassifiseres som *unknown* og dermed forkastes før de når spillet. Augmenteringen i E2 gir samtidig økt robusthet mot variasjon i mikrofon, romakustikk og volum. Tap i benchmark-ytelse (test macro F1 70,7 % mot 4-klasse-baseline 83,5 %) er en bevisst avveining til fordel for praktisk robusthet.

Konfidensterskel på 0,55 komplementerer *unknown* klassen ved å filtrere prediksjoner der modellen er usikker selv om en kommandoklasse ble valgt. Verdien er en avveining: for lav terskel gir falske positive (slangen svinger uventet ved bakgrunnsstøy), for høy verdi gir falske negative (tydelige kommandoer forkastes og spillet føles ikke-responsivt). 0,55 er empirisk identifisert som et akseptabelt kompromiss.

5.3 Begrensninger og videre arbeid

Selv med *unknown* klasse er det en observerbar ytelsesforskjell mellom testsett og live mikrofonbruk, ettersom Speech Commands ikke fullt ut representerer reell romakustikk og personlig talemønster. *Unknown* klassen i datasettet består dessuten av andre talte ord, ikke stillhet eller miljøstøy, og fanger derfor ikke opp alle typer irrelevant input i reell bruk. En naturlig videreføring er å supplere treningssettet med eksplisitte stillhets- og bakgrunnsstøyopptak.

Applikasjonen er forberedt for distribusjon på Hugging Face Spaces med dedikert app.py, requirements.txt og README. Live fanen bruker kun «left» og «right» fordi det speiler den klassiske tastaturstyringen av Snake, der pilene høyre og venstre dreier slangen relativt til nåværende retning. Dette gir et mer naturlig og umiddelbart grensesnitt for sanntidsstyring. Mulige videre forbedringer er en adaptiv konfidensterskel som justeres basert på rullende støynivå, eksplisitt warmup ved oppstart for å eliminere first-run overhead i PyTorchs JIT-kompilering, og en utvidelse av treningssettet med eksplisitte stillhets- og bakgrunnsstøyopptak for økt robusthet i live bruk.

6 Konklusjon

Dette prosjektet undersøkte om et kompakt, lokalt CNN-basert keyword spotting system kunne trenes fra bunnen av og brukes til stemmestyring av et Snake-spill i sanntid. Resultatene viser at svaret i hovedsak er ja. 4-klasse-baselinen oppnådde 83,9 % test-accuracy og 83,5 % macro F1, mens den deployerte E2 modellen (5-klasse, downsampling + augmentering) oppnådde 78,4 % accuracy og 70,7 % macro F1, men håndterer irrelevant tale eksplisitt gjennom *unknown* klassen.

Eksperimentene viste også at oppgaven blir tydelig mer krevende når unknown inkluderes som egen klasse, at downsampling og audioaugmentering forbedret robustheten, og at den spektrogrambaserte CNN-modellen presterte klart bedre enn modellen som brukte rå waveform.

Prosjektets viktigste begrensninger er gapet mellom benchmark-data og reell mikrofonbruk, fraværet av en egen silence-klasse, og at vokabularet er begrenset til engelske kommandoer. Videre arbeid bør derfor prioritere en mer realistisk datainnsamling, inkludering av silence og testing på norske taledata. Prosjektet bekrefter at dyp læring er en egnet metode for begrenset talegjenkjenning, men også at praktisk bruk krever mer enn høy testytelse: robust evaluering og håndtering av irrelevante signaler er avgjørende for at systemet skal fungere utenfor et kontrollert benchmark.

7 Kilder

Chollet, F., og Watson, M. (2026). Deep learning with Python (3. utg.). Manning Publications.

Goodfellow, I., Bengio, Y., og Courville, A. (2016). Deep learning. MIT Press.

Gradio. (2023). Gradio documentation. <https://gradio.app/docs/>

HuggingFace. (2023). datasets: google/speech_commands.
https://huggingface.co/datasets/google/speech_commands

Kingma, D. P., og Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/1412.6980>

PyTorch. (2023). PyTorch documentation. <https://pytorch.org/docs/stable/index.html>

Rybakov, O., Kononenko, N., Subrahmanya, N., Visontai, M., og Lorenzo, S. (2020). Streaming keyword spotting on mobile devices. Interspeech 2020, 2277–2281.
<https://doi.org/10.21437/Interspeech.2020-1003>

Sainath, T. N., og Parada, C. (2015). Convolutional neural networks for small-footprint keyword spotting. Interspeech 2015, 1478–1482. <https://doi.org/10.21437/Interspeech.2015-352>

Zhang, P., og Zhang, X. (2020). Deep template matching for small-footprint and configurable keyword spotting. Interspeech 2020, 2572–2576. <https://doi.org/10.21437/Interspeech.2020-1761>

Stevens, S. S., Volkman, J., & Newman, E. B. (1937). A scale for the measurement of the psychological magnitude pitch. The Journal of the Acoustical Society of America, 8(3), 185–190. <https://doi.org/10.1121/1.1915893>

torchaudio. (2023). torchaudio documentation. <https://pytorch.org/audio/stable/index.html>

Warden, P. (2018). Speech commands: A dataset for limited-vocabulary speech recognition. arXiv preprint arXiv:1804.03209. <https://arxiv.org/abs/1804.03209>

8 Vedlegg

8.1 Erklæring om bruk av kunstig intelligens (KI)

Først av alt, les HVL sin [veileder for bruk av kunstig intelligens](#).

Kandidatnummer	423 - 437 - 366 - 361
Semester	Vår 2026

Har det i arbeidet med denne rapporten blitt anvendt KI-baserte hjelpemidler?

☒ Ja

☐ Nei

Hvis ja, spesifiser type verktøy og bruksområde nedenfor:

Tekst

☒ Stavekontroll: Er deler av teksten kontrollert av KI-verktøy?

☐ Skriveassistanse: Er deler av strukturen, ideene eller fremgangsmåtene basert på forslag fra KI-verktøy?

☐ Tekstgenerering: Er deler av teksten generert av KI-verktøy? (f.eks. ChatGPT)

Hvis ja, spesifiser hvilke verktøy og hva dere har brukt de til her:

Brukt ChatGPT for stavekontroll og hjelp til formulering av vanskelige setninger.

Prosess

☒ Idégenerering: Har KI-verktøy blitt brukt til idégenerering i arbeidet med prosjektet?

☒ Sparringspartner: Har KI-verktøy blitt brukt som sparringspartner under prosjektet?

(F.eks. for å forstå konsepter fra pensum og forelesninger eller for å få tilbakemeldinger på idéer.)

☐ Informasjonssøk: Har KI-verktøy blitt brukt til informasjonssøk under prosjektet?

Hvis ja, spesifiser hvilke verktøy og hva dere har brukt de til her:

Brukt ChatGPT 5.2 som veiledning til mappestruktur, kodeforslag i notebookene og fått forslag til hva som skal inkluderes i repoet.

Bilder og figurer

Bildegenerering: Har ett eller flere av bildene eller figurene i rapporten blitt generert av KI-verktøy?

Hvis ja, spesifiser hvilke verktøy og hva dere har brukt de til her:

Annet

☐ Andre KI-verktøy

Spesifiser hvilke verktøy og hva dere har brukt de til her: